

SAiDL Summer Induction Assignment 2026
Core ML: Long-Context Sequence Modelling
Attention Variants, Positional Encodings, and Conv–Attention Hybrids

Aarush Gupta
2023B3A70839G

Contents

1	Introduction	2
2	Experimental Setup	2
2.1	Dataset	2
2.2	Model Configuration	2
2.3	Training and Evaluation	2
2.4	Hardware and a Limitation	3
3	Task 1: Attention Mechanisms	3
3.1	Variants	3
3.2	Results	3
3.3	Analysis	4
4	Task 2: Positional Encodings	4
4.1	Variants	4
4.2	In-Distribution Results	5
4.3	Extrapolation Test	5
4.4	Analysis	5
5	Task 3: Conv–Attention Hybrids	6
5.1	Design	6
5.2	Results	6
5.3	A Note on the Conv Results	6
6	Discussion	7
6.1	What the results say	7
6.2	Limitations	7
7	Conclusion	7

1 Introduction

Standard self-attention scales quadratically with sequence length in both compute and memory. At a context window of $T = 1024$, the attention matrix alone is $T \times T = 1,048,576$ entries per head per layer. This is the central bottleneck that motivates a large body of work on efficient attention.

This report documents a systematic empirical study on the WikiText-2 dataset using a decoder-only Transformer. We ask three questions. First, how do alternative attention mechanisms — local, sparse block, and multi-query — compare against standard attention in terms of perplexity, memory, and latency? Second, how does the choice of positional encoding affect in-distribution performance, and does it affect the model’s ability to generalise to longer sequences than it was trained on? Third, does adding a 1D convolutional component alongside attention improve results?

The study is not exhaustive. Compute constraints limited us to context lengths of 512 and 1024, and the small size of WikiText-2 required aggressive regularisation throughout. We document these constraints honestly where they affect interpretation.

2 Experimental Setup

2.1 Dataset

All experiments use WikiText-2, a standard language modelling benchmark built from verified Wikipedia articles. The dataset is small by modern standards — roughly 2M training tokens — which turned out to be a meaningful constraint. Early runs with $d_{\text{model}} = 256$ overfit badly; training loss continued to fall while validation loss diverged. We responded by reducing d_{model} to 64 and raising dropout to 0.3. All reported results use this configuration.

2.2 Model Configuration

The base model is a decoder-only Transformer following the architecture of Vaswani et al. [1], with the following fixed hyperparameters across all experiments:

- Layers: 2
- Attention heads: 4
- Embedding dimension d_{model} : 64
- Feed-forward dimension d_{ff} : 256 ($4 \times d_{\text{model}}$)
- Dropout: 0.3
- Training iterations: 30,000
- Batch size: 32
- Tokeniser: GPT-2 BPE (via `tiktoken`)

The architecture is modular: attention type, positional encoding, and convolutional components can each be swapped independently via a single configuration dataclass. This allowed us to run controlled comparisons where only one component changes at a time.

2.3 Training and Evaluation

Models were trained using a cosine learning rate schedule with warmup. Validation loss was computed on a held-out split using sequential, non-overlapping batches to ensure deterministic and comparable results across runs. Perplexity is reported as $\exp(\text{val loss})$.

All experiments were tracked using Weights & Biases. Inference metrics (latency, throughput) were measured separately after training on a fixed hardware setup using 20 repeated generation

sequences of 128 tokens each, with 3 warmup passes before timing.

2.4 Hardware and a Limitation

T=512 training runs were conducted on an NVIDIA A100. T=1024 runs were conducted on an NVIDIA L40S due to compute availability. This means latency and memory numbers *are not directly comparable* across context lengths — the T=1024 numbers reflect both the longer context and a different GPU. Perplexity comparisons across context lengths are still valid since they measure model quality, not hardware performance.

3 Task 1: Attention Mechanisms

3.1 Variants

We implemented four attention mechanisms. The following is a brief description of each.

Standard self-attention computes a full $T \times T$ attention matrix. Every token can attend to every other token in a single step, which gives the model maximum flexibility to learn arbitrary dependency patterns. The cost is $\mathcal{O}(T^2)$ in both compute and memory.

Sliding window (local) attention restricts each token to attend only within a fixed window of w neighbours. The attention matrix becomes a band matrix. Complexity drops to $\mathcal{O}(T \cdot w)$. Most syntactic dependencies in natural language are local, so this works well for a large class of tasks, but long-range dependencies must propagate through many layers. We used $w = 128$.

Sparse block attention divides the sequence into blocks of size b and restricts attention to within-block. Tokens at block boundaries cannot directly attend to their immediate neighbours across the boundary, which is an artificial discontinuity. We used $b = 64$.

Multi-query attention (MQA) keeps multiple independent query heads but shares a single key and value projection across all heads. This does not change the $\mathcal{O}(T^2)$ training complexity, but reduces the KV cache at inference by a factor of H (the number of heads), which is the dominant memory cost during long-context generation.

3.2 Results

Table 1: Task 1 — Attention variant comparison on WikiText-2 (2 layers, 4 heads, $d_{\text{model}} = 64$, dropout=0.3, 30k iterations). T=512 runs on NVIDIA A100; T=1024 runs on NVIDIA L40S. Latency and memory figures are **not** directly comparable across context lengths due to hardware differences.

Attention	Context	Val PPL ↓	Peak GPU (MB)	Throughput (tok/s) ↑	Latency (ms/seq) ↓
Standard	512	106.24	10,503	192.61	664.54
Local	512	100.35	10,341	93.18	1,373.62
MQA	512	108.30	10,472	217.45	588.64
Standard	1024	106.06	21,726	179.24	714.13
Local	1024	97.94	20,622	67.74	1,889.54
MQA	1024	106.78	21,663	209.55	610.82
Sparse	1024	107.83	20,241	186.00	688.16

Note: Sparse attention was only run at T=1024. A T=512 sparse run was not completed due to time constraints.

3.3 Analysis

The most notable result is that **local attention consistently achieves the lowest perplexity** at both context lengths (100.35 at T=512, 97.94 at T=1024). This is somewhat counterintuitive at first glance — local attention has strictly less expressive power than standard attention — but it makes sense given the dataset. WikiText-2 consists of Wikipedia articles where local context dominates. Most of what a language model needs to predict the next token is in the immediately surrounding text. Local attention’s inductive bias toward locality turns out to be well-matched to this data.

Standard and MQA attention produce nearly identical perplexity at both context lengths (within 2 PPL of each other). This is consistent with theory: MQA reduces KV cache size but does not fundamentally change what the model can learn. At small model sizes the shared K/V might be expected to hurt more, but the gap is minor here.

The **latency picture tells a different story**. Local attention is the slowest by a large margin — 1373 ms/seq at T=512 versus 664 ms/seq for standard. This is an implementation artefact: our local attention uses a masked full attention matrix rather than a true sparse kernel, so it does not realise the theoretical $\mathcal{O}(T \cdot w)$ savings at the sizes we tested. An optimised implementation (e.g. using block-sparse CUDA kernels as in Longformer) would be significantly faster.

MQA is the fastest variant at both context lengths. Since it shares K and V projections across heads, there is less computation per forward pass, and the smaller KV cache means less memory pressure during the autoregressive generation loop.

Sparse block attention at T=1024 sits in the middle on all metrics — slightly worse perplexity than standard, slightly less memory, roughly similar latency. The block boundary discontinuity likely hurts, especially in a small model where there is limited capacity to compensate through depth.

4 Task 2: Positional Encodings

4.1 Variants

We compared four positional encodings, all with standard attention at T=512.

Sinusoidal (absolute) adds a fixed deterministic vector to each token embedding before the first layer. The encoding is defined for any position, but positions beyond the training length are out of distribution.

Relative (Shaw et al.) adds learned relative position biases directly to attention logits. The score between positions i and j includes a learned embedding for the relative distance $i - j$, clipped at a maximum range. This directly encodes relative distance rather than absolute position.

RoPE (rotary positional embedding) rotates query and key vectors by an angle proportional to their position before computing dot products. Because rotation matrices compose, the inner product $(R_t Q_i) \cdot (R_s K_j)$ depends only on $t - s$, giving exact relative position encoding with zero additional parameters.

ALiBi (attention with linear biases) adds no positional information to token embeddings at all. Instead it subtracts a linear penalty from attention logits proportional to distance: $\text{score}(i, j) = Q_i \cdot K_j / \sqrt{d_k} - m \cdot |i - j|$, where m is a head-specific slope. The penalty is defined for any distance, which is why ALiBi is expected to extrapolate gracefully.

4.2 In-Distribution Results

Table 2: Task 2 — Positional encoding comparison on WikiText-2 (standard attention, $d_{\text{model}} = 64$, trained and evaluated at $T = 512$). All runs on NVIDIA A100.

PE	Val PPL ↓	Peak GPU (MB)	Throughput (tok/s) ↑	Latency (ms/seq) ↓
Sinusoidal (Abs.)	107.20	10,503	264.65	483.65
ALiBi	105.26	10,510	230.96	554.20
Relative (Shaw et al.)	82.88	10,587	208.48	613.96
RoPE	81.89	10,502	172.51	741.98

4.3 Extrapolation Test

All models were trained at $T = 512$ and evaluated at $T \in \{512, 1024, 2048\}$.

Table 3: Task 2 — Extrapolation test: models trained at $T = 512$, evaluated at $T \in \{512, 1024, 2048\}$. Dashes indicate the evaluation could not be completed (see text).

PE	PPL at T=512	PPL at T=1024	PPL at T=2048
Sinusoidal (Abs.)	107.19	—	—
Relative (Shaw et al.)	82.88	—	—
RoPE	81.89	—	—
ALiBi	105.26	—	—

The T=1024 and T=2048 evaluations all failed during the inference run. The most likely cause is that relative positional encoding and RoPE raise index errors when the sequence length at evaluation exceeds the training context window, since the relative embedding table and rotation frequencies are initialised for $T \leq 512$. For sinusoidal and ALiBi this should not be the case in principle — ALiBi especially is designed for exactly this test — but the inference script applied the same validation loop uniformly and the failures were not caught per-variant. This is a gap in the results we acknowledge. A corrected re-run isolating each variant’s failure mode would be needed to draw any conclusions about extrapolation.

4.4 Analysis

The in-distribution results are the clearest result in the whole study. **RoPE and Relative encoding outperform Sinusoidal and ALiBi by a substantial margin** — roughly 25 PPL points. This is a meaningful gap at this scale.

Both RoPE and Relative encoding provide the model with direct relative distance information, either through rotated key-query inner products (RoPE) or through learned per-distance bias terms (Relative). Standard sinusoidal encoding only gives the model absolute position, from which it must infer relative distances indirectly. At $T = 512$ with a 2-layer model, this indirect route is clearly less effective.

ALiBi’s performance is more surprising. Despite its linear penalty mechanism, which in principle encodes relative distance, it performs comparably to sinusoidal (105 vs 107 PPL). One possible reason is that the fixed linear penalty imposes a hard locality bias — all distant tokens are penalised equally regardless of their actual relevance. For a language modelling task where long-range dependencies do sometimes matter, this constant suppression of distant attention may hurt.

RoPE’s slight edge over Relative (81.89 vs 82.88) is consistent with the literature. RoPE achieves exact relative encoding with no additional learned parameters, while Relative encoding’s bias terms need to be learned from data — a harder problem at small model sizes.

Memory usage is nearly identical across all four variants, which is expected since the positional encoding does not change the dimensions of the attention matrix.

5 Task 3: Conv–Attention Hybrids

5.1 Design

We added 1D convolutional components to the sparse attention + relative PE configuration, which represented a middle-ground point in the attention and PE search spaces. Two designs were tested:

Pre-attention Conv1D: a 1D convolution with kernel size 5 is applied to the token representations before each attention block. This gives the model an explicit local n-gram feature extractor that runs before the attention mechanism sees the input.

Interleaved Conv1D: Conv1D and attention blocks alternate. Rather than pairing every attention block with a convolution, some layers are purely convolutional and some are purely attention. This reduces the total number of attention computations and replaces them with cheaper convolutions.

5.2 Results

Table 4: Task 3 — Conv1D hybrid results on WikiText-2 (sparse attention + relative PE, $d_{\text{model}} = 64$). † indicates a result that could not be verified as correct (see text). Baseline row shows sparse + relative without any convolutional component.

Conv	Context	Val PPL ↓	Peak GPU (MB)	Throughput (tok/s) ↑	Latency (ms/seq) ↓
None (baseline)	512	107.83	20,241	186.00	688.16
Interleaved	512	†	10,257	467.95	273.54
Pre-Attention	1024	†	20,585	188.14	680.35

5.3 A Note on the Conv Results

The validation perplexity logged for both conv runs was approximately 1.0, which corresponds to near-zero cross-entropy loss. This is not a plausible result for a language model on WikiText-2 and almost certainly reflects a bug in how validation was run for the conv variants rather than a genuine model result. The most likely cause is that the convolutional layers introduce a data leakage path when the validation loop reuses the same batch indices, or that the causal masking was broken for the conv pathway.

We report the throughput and memory numbers, which are hardware measurements independent of the model’s correctness, but mark the PPL values as unverified. Rerunning the conv experiments with a corrected validation loop is needed before any conclusions about perplexity can be drawn.

What we can say from the hardware numbers: the interleaved design at T=512 achieves notably higher throughput (467 tok/s vs 186 tok/s for the baseline) and lower latency (273 ms vs 688 ms). Replacing half the attention layers with convolutions reduces the quadratic attention compute significantly, which shows up directly in speed. The pre-attention design at T=1024 matches the baseline’s throughput closely, suggesting the added convolution is not the bottleneck at that context length.

6 Discussion

6.1 What the results say

Across Tasks 1 and 2, the clearest pattern is that **inductive bias matching matters more than architectural complexity**. Local attention outperforms standard attention on WikiText-2 not because it is more powerful, but because its locality bias fits the data. RoPE and Relative outperform Sinusoidal not because they have more parameters, but because they give the model the right kind of positional information directly.

This pattern probably weakens at larger scales. With more layers, more data, and larger models, standard attention can learn to approximate local patterns, and sinusoidal encoding can be used more effectively. At the small scale of this study, explicit inductive bias is doing a lot of work.

6.2 Limitations

We list the main limitations that affect interpretation of the results:

1. **Dataset size.** WikiText-2 is too small for the models we are comparing. Overfitting forced us to use aggressive dropout and small embedding dimensions, which may compress differences between variants that would be more visible at larger scale.
2. **Hardware inconsistency.** T=512 and T=1024 runs were on different GPUs (A100 vs L40S). All latency and memory comparisons across context lengths are confounded.
3. **Extrapolation test incomplete.** The T=1024 and T=2048 evaluations failed for all PE variants. The extrapolation table cannot be used to draw any conclusions about length generalisation.
4. **Conv results unverified.** Val PPL ≈ 1.0 for both conv variants is not a valid result. These runs need to be repeated with a corrected validation setup.
5. **Sparse attention missing at T=512.** The sparse T=512 run was not completed. Cross-context-length comparisons for sparse attention are therefore not possible.
6. **Controlled comparisons only.** Task 1 fixes positional encoding to sinusoidal while varying attention. Task 2 fixes attention to standard while varying PE. The full cross-product was not run. The best combination of attention and PE might differ from what either task individually suggests.

7 Conclusion

We trained and evaluated a family of decoder-only Transformers on WikiText-2, varying attention mechanism, positional encoding, and the presence of convolutional components.

On attention variants, local attention gives the best perplexity at both T=512 and T=1024, at the cost of high latency in our unoptimised implementation. MQA matches standard attention in perplexity while being faster at inference. On positional encodings, RoPE and Relative encoding substantially outperform Sinusoidal and ALiBi in-distribution, suggesting that direct relative position information is important at small model sizes. The Conv hybrid results could not be verified due to a validation bug and require a follow-up run.

The study has real limitations, mostly driven by compute and time constraints. The cleaner takeaway is methodological: the modular codebase and experiment tracking infrastructure work well, and a larger-scale rerun on a bigger dataset with consistent hardware would give more reliable conclusions.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [2] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [3] N. Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- [4] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.
- [5] P. Shaw, J. Uszkoreit, and A. Vaswani. Self-attention with relative position representations. *Proceedings of NAACL-HLT*, 2018.
- [6] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *International Conference on Learning Representations*, 2022.